Multitask policy learning and Vision-Language-Action models *continued…*

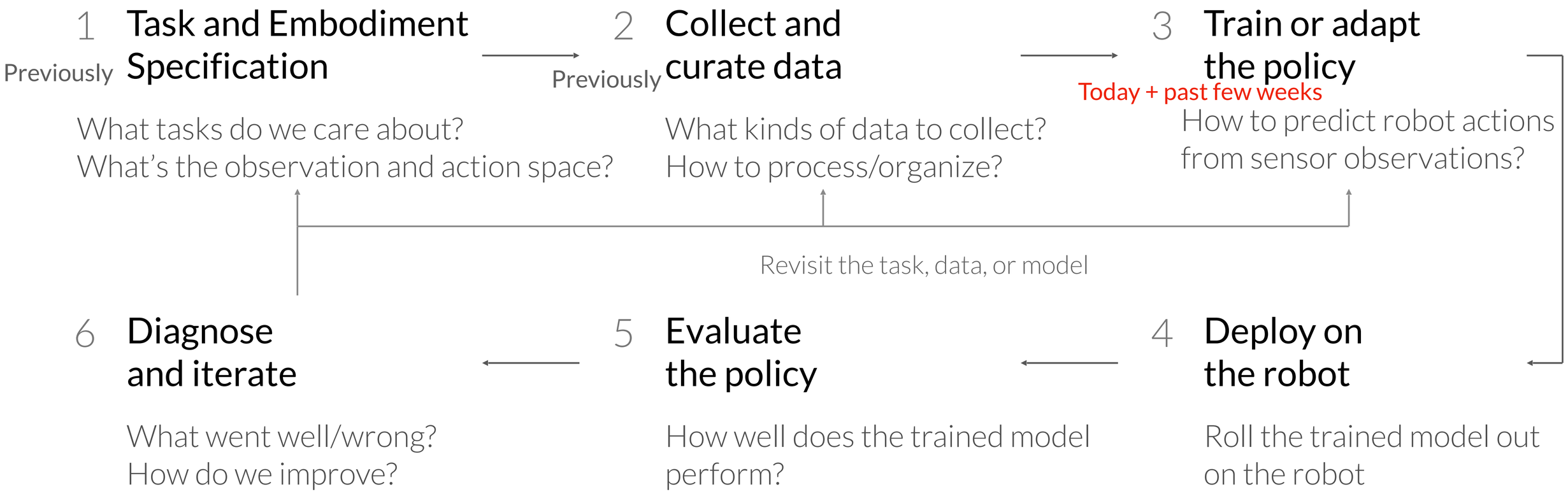
EN.601.[4|6]98 Hands-on Robot Learning

Krishna Murthy Jatavallabhula

Administrivia

- Shipping delays (as has been the norm this semester…)
- Assignment 1 and Assignment 2 deadlines updated (some breathing room)
- Today's lecture is split into two parts:
  - First, we will finish up our discussion of VLA models
  - Then, course staff will help teams prep for assignment 1 demos (due in class, next Thursday — 5 November)
- Next week (5 Nov):
  - First half will be a lecture — agentic approaches to robot learning
  - Second half will be demos and evaluations for assignment 1

The *modern* robot learning lifecycle



Recap: *Generalization* in robot learning

- In single-task imitation learning, the *task* is *implicitly specified* (by the demonstration distribution)
- To build robots that *generalize*, it is infeasible to collect demonstrations and train a task-specific policy for every new task the robot will ever encounter
- We would like one policy that can perform many tasks
- *Generalization* can take on multiple meanings — new objects, environments, robot embodiments, tasks; or *all of these*

Recap: Early VLAs

- Early VLAs relied largely on the capabilities of a base VLM policy for generality
- Most of the demos (pick-place tasks) focus on generalization at the semantic level (i.e., new objects in new settings; new language instructions)
- Limited complexity in robot actions (which necessitated skill selection, hierarchies, sketches, two-level action decoding, etc.)
- The next generation of VLA models that followed the RT series (and which we'll see next) go the opposite way (emphasize on large-scale imitation learning to generate more complex robot behaviors; often at the expense of semantic generalization)

Recap: Key hurdles/issues* with generalist policies

- Last week
  - Problem 1: How do we effectively represent robot data for policy learning?
    - Tokenization methods (we will see FAST, VQ-BeT, OAT), pi0
  - Problem 2: How do we efficiently learn from multiple data sources?
    - NVIDIA GR00T N1 / N1.5 models (latent actions unify data sources)
- Today
  - Problem 3: How do we retain web-scale pretrained knowledge?
    - pi0.5, knowledge insulation
  - Problem 4: How do we learn new tasks efficiently?
    - Few-shot and In-context learning

* Non-exhaustive list

Problem 3: how do we retain web-scale pretrained knowledge?

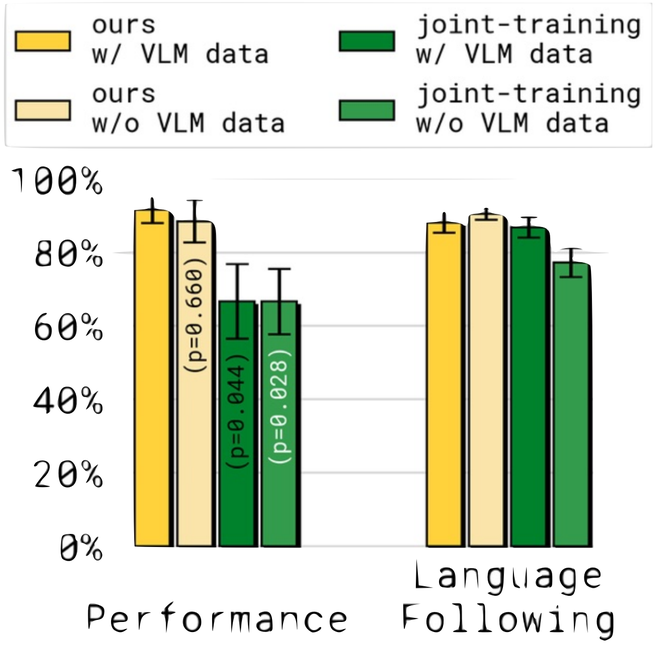
- Web-scale (VLM) pretraining provides broad visual, semantic, and linguistic priors
- VLA models should ideally leverage all these capabilities to generalize to newer objects, environments, and tasks
- But, naive finetuning / re-training of a VLM to obtain a VLA leads to *catastrophic forgetting* (i.e., underlying VLM modules *forget* much of what was learned before)
- One could keep the VLM frozen (every downstream module conditions on the frozen VLM embeddings)
- However, this (naive) approach doesn't work (representations need to evolve to be predictive of robot actions)
- pi0.5 addresses it with what they term *knowledge insulation*

Zitkovich et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. CoRL. 2023.

Driess et al. Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better. NeurIPS. 2025.

It is a bad idea to *just initialize* an action expert on top of a VLM and train it

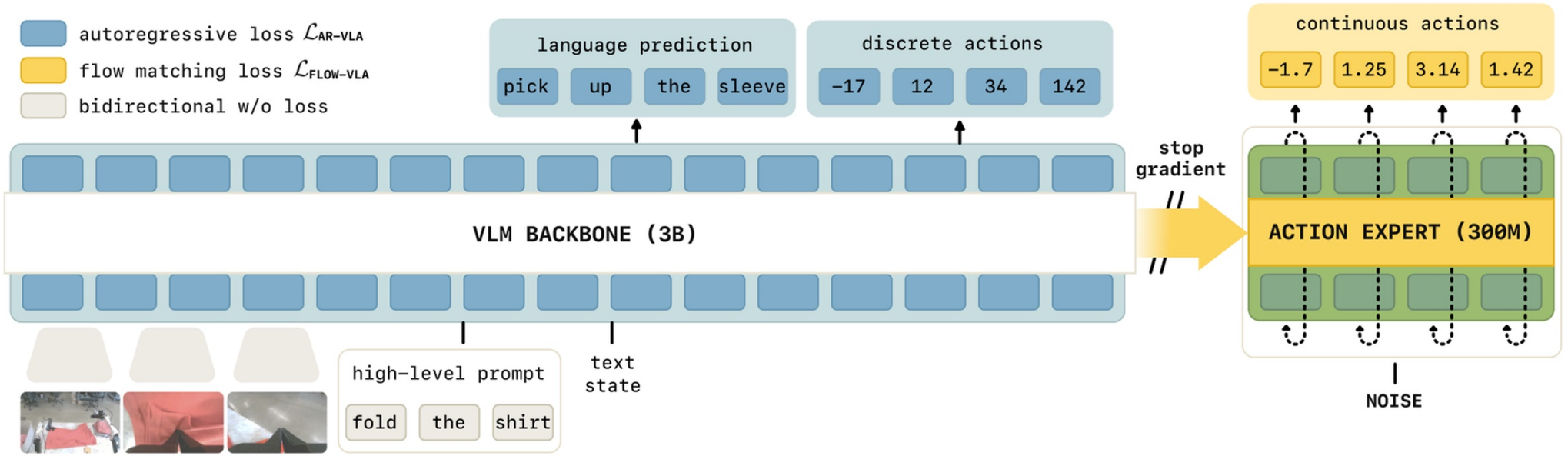
- Consider a pristine, frontier VLM that just cost a billion $$$ to train
- It is tempting to strap a *randomly initialized* action expert (a flow matching head) on top and train it with robot data
- A *very bad* idea!
- The action expert is initially (inevitably) going to produce poor actions predictions, which means gradients through it will be large
- We do not want these flowing back through the VLM and resulting in catastrophic forgetting
- At the same time, we also don't want to just freeze the VLM



Driess et al. Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better. NeurIPS. 2025.

Knowledge insulation

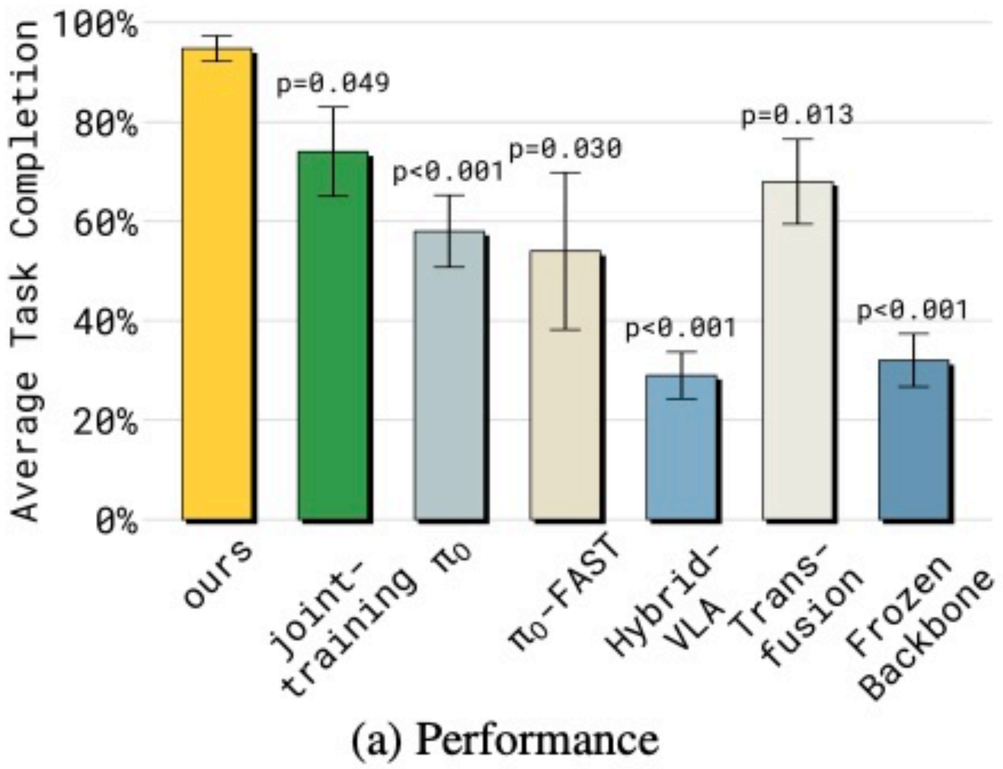
- Key idea 1: Co-train on generally robotics relevant vision-language tasks
- Use those gradients to keep updating the VLM backbone
- Use (continuous) action predictions to only update only the action expert
- Key idea 2: Add discrete action prediction (FAST tokenization) as auxiliary objective to update VLM backbone



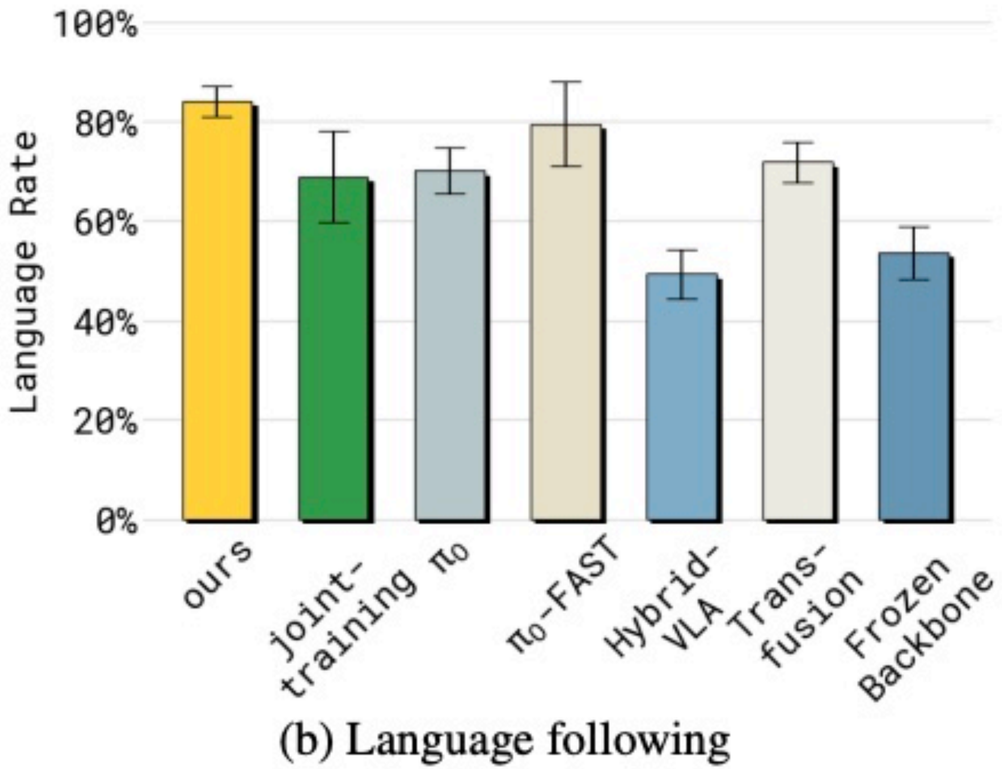
Driess et al. Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better. NeurIPS. 2025.

Knowledge insulation: results

- Compare joint-training, frozen backbone, and knowledge insulation
- Metrics: Instruction following, task performance and (not shown) training speed



(a) Performance

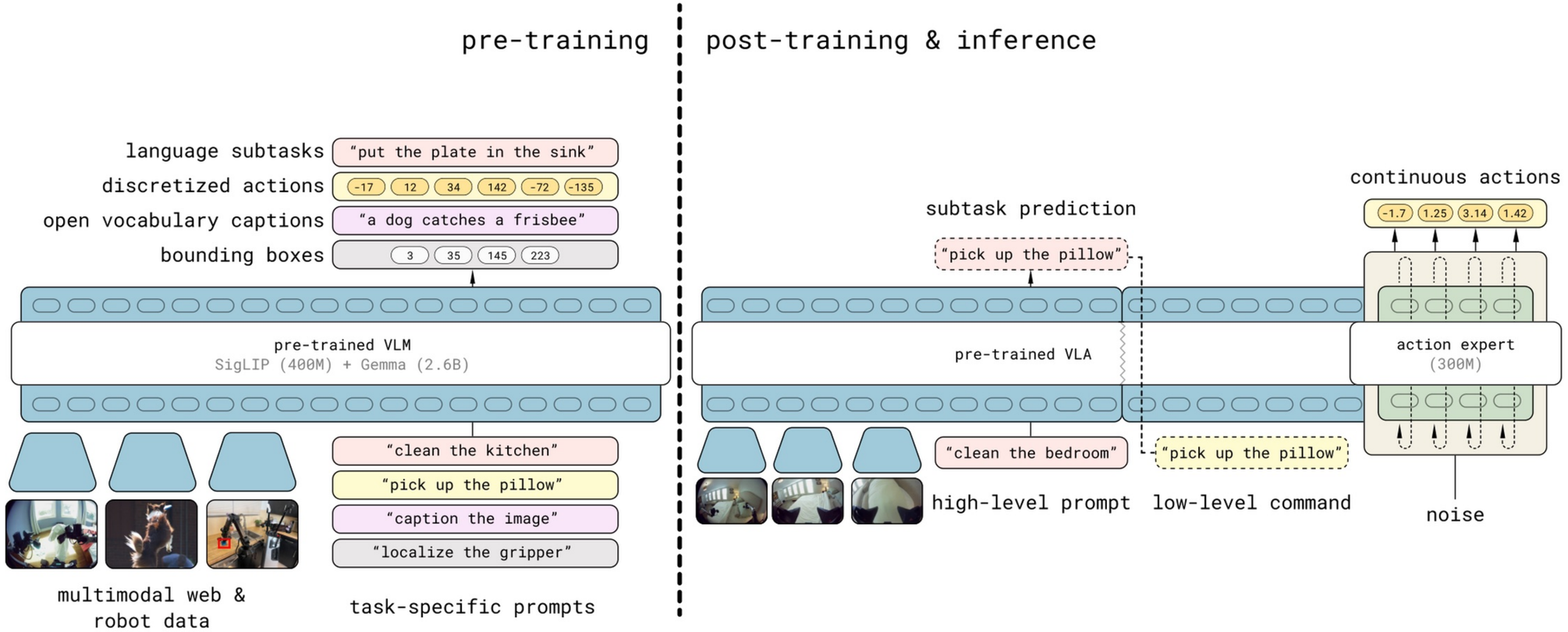


(b) Language following

Driess et al. Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better. NeurIPS. 2025.

pi0.5: architecture and training

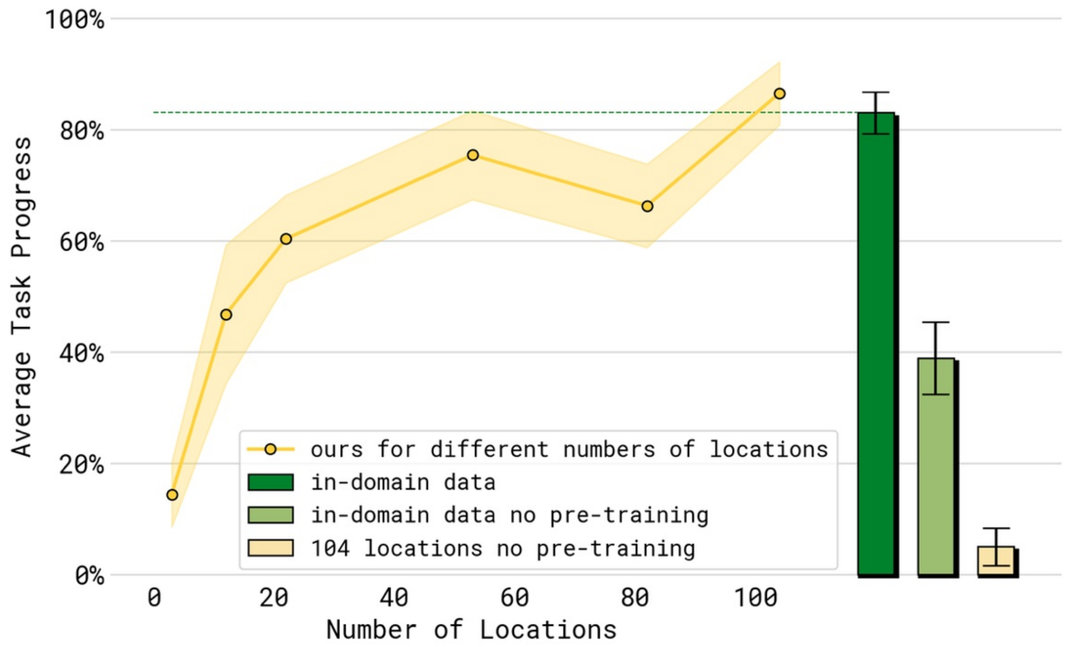
- Predict a subtask, e.g., “put the plate in the sink,” from the overall instruction and scene
- Condition low-level action generation on this subtask and current observations



Black et al. pi0.5: a Vision-Language-Action Model with Open-World Generalization. CoRL. 2025.

pi0.5: deployment in new homes

- Evaluate household tasks in homes absent from the training data
- Training-location diversity improves transfer to held-out environments



Black et al. pi0.5: a Vision-Language-Action Model with Open-World Generalization. CoRL. 2025.

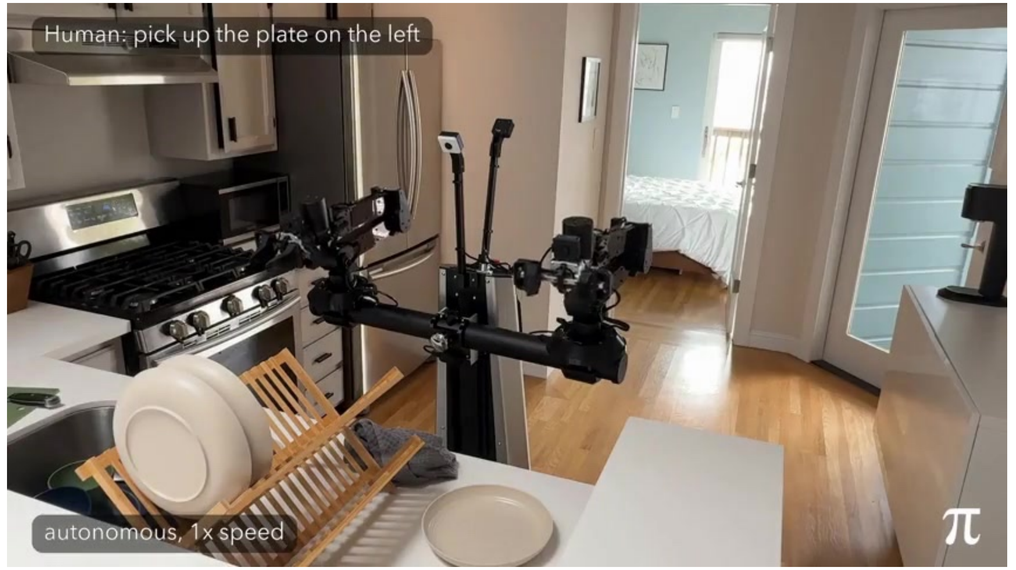
pi0.5: execution in a held-out home

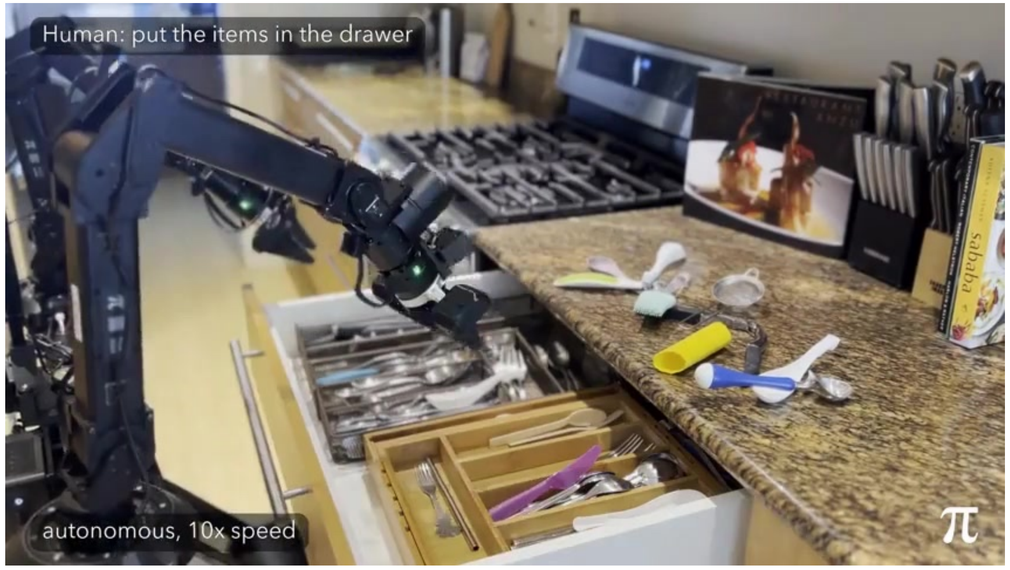


Black et al. pi0.5: a Vision-Language-Action Model with Open-World Generalization. CoRL. 2025.

pi0.5 (more visuals)



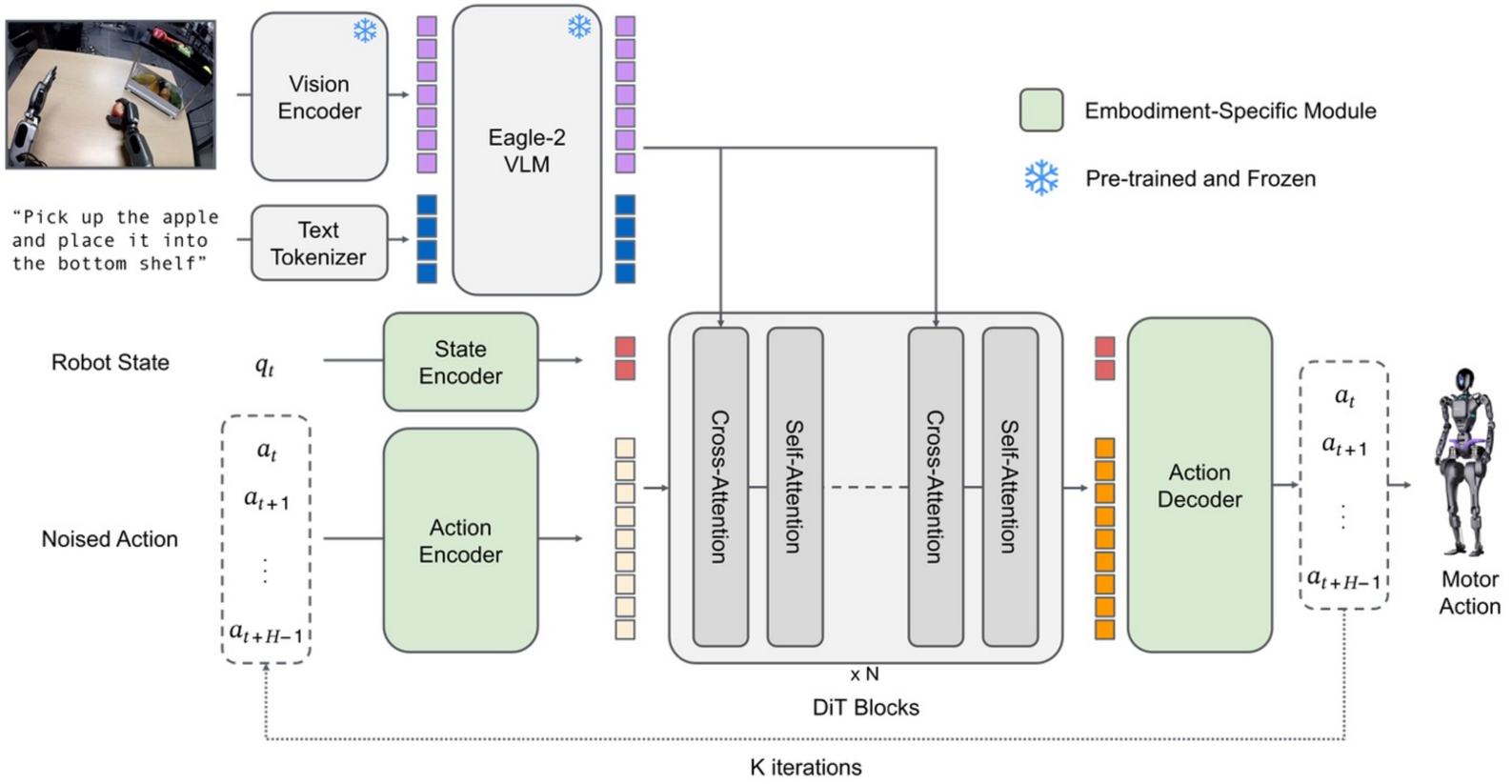






GR00T N1.5: freezing the VLM

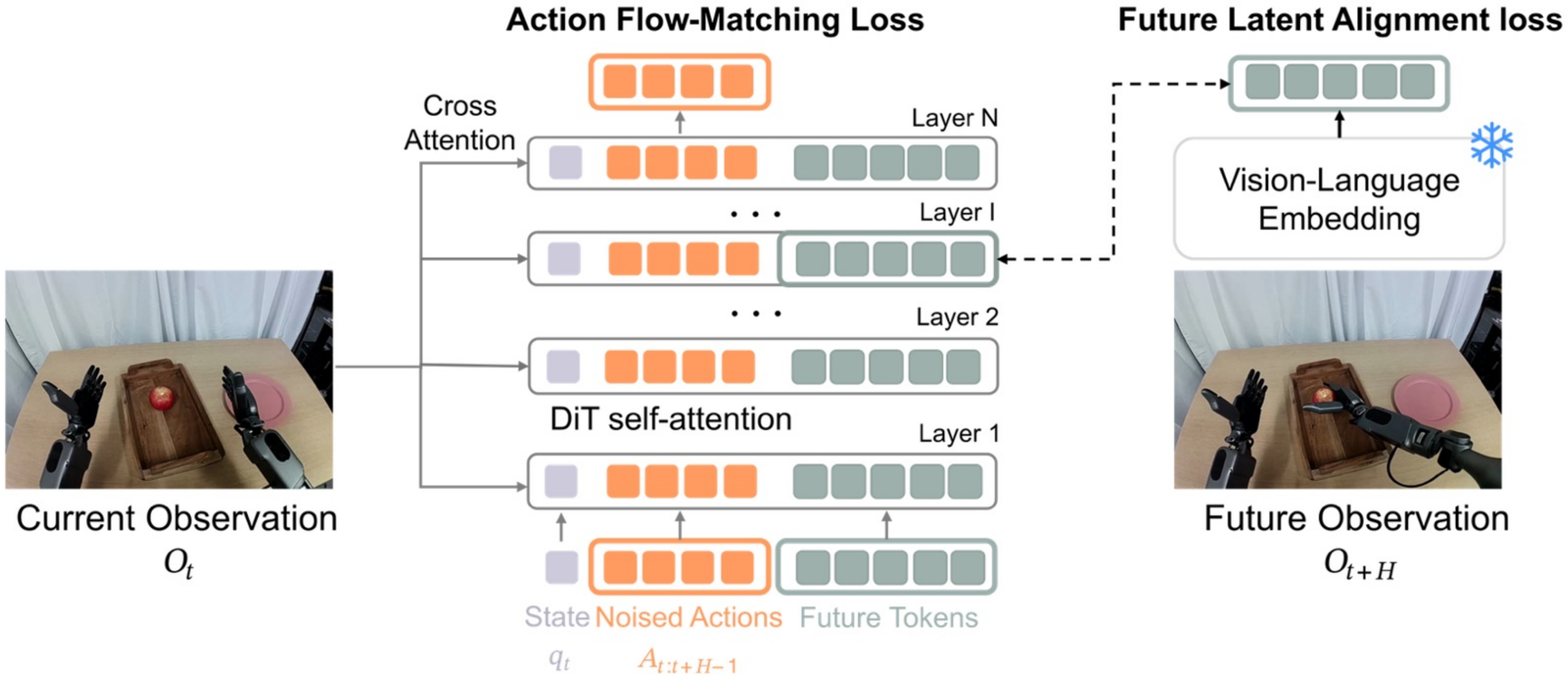
- Freeze the VLM during policy pretraining and fine-tuning
- Blocks action gradients completely, unlike knowledge insulation
- However, the action expert is now trained with a FLARE (future representation prediction) objective



NVIDIA. GR00T N1.5. Technical release. 2025.

FLARE: predicting future features

- Add a prediction target for latent features of future observations
- Actionless human video contributes to improved performance
- This objective forms the basis for modern world action models (WAMs), which we'll see in a later lecture

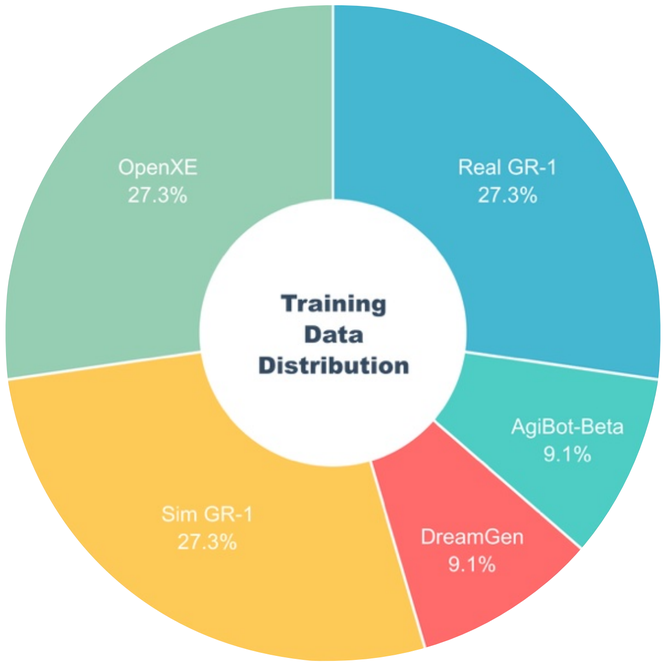


Zheng et al. FLARE: Robot Learning with Implicit World Modeling. CoRL. 2025.

NVIDIA. GR00T N1.5. Technical release. 2025.

GR00T N1.5: data and supervision

- Combine robot trajectories, simulation augmentation, generated examples and human video
- DreamGen adds neural trajectories
- FLARE incorporates a future observation encoding prediction objective



NVIDIA. GR00T N1.5. Technical release. 2025.
Jang et al. DreamGen: Unlocking Generalization in Robot Learning through Video World Models. CoRL. 2025.
Zheng et al. FLARE: Robot Learning with Implicit World Modeling. CoRL. 2025.

Impact of fine-tuning on previously-learned skills

- Fine-tuning can also change/remove skills acquired during robot pretraining
- E.g., a useful handover behavior in the base policy is lost after fine-tuning

Task: `Pick up red apple and place it into the basket`

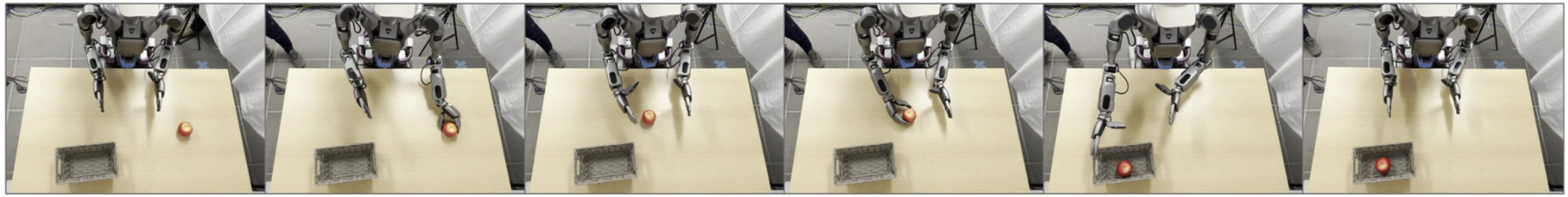
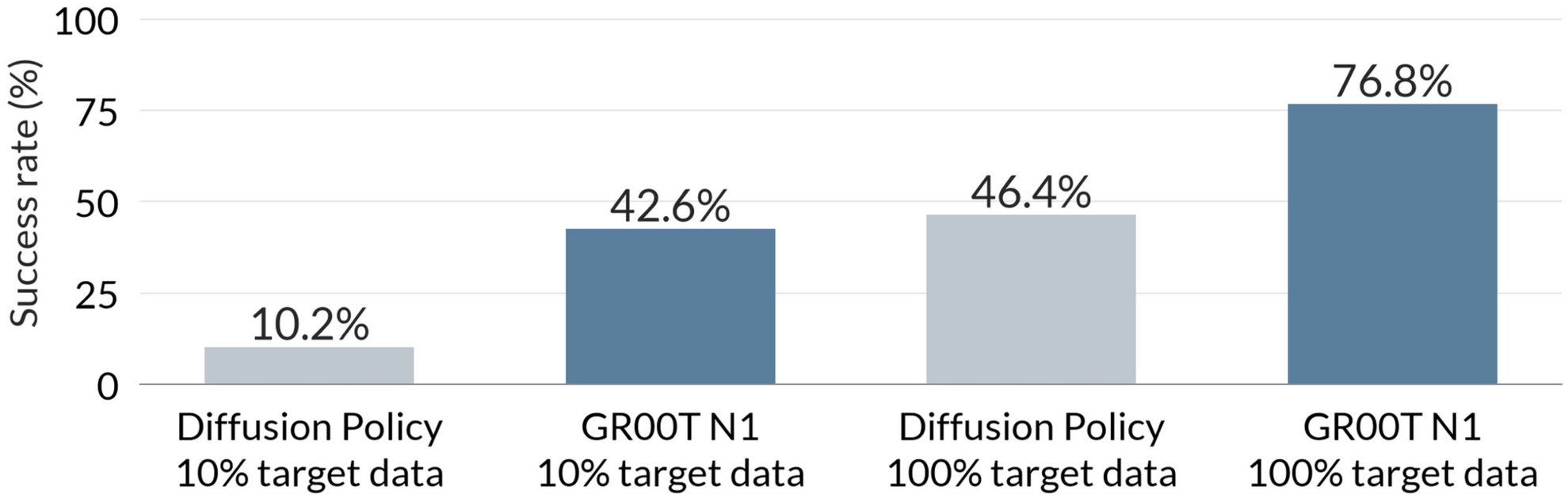


Figure 11: **Pre-training Qualitative Example.** While prompting the pretrained GR00T-N1-2B model with a post-training task instruction, we even increase the difficulty by placing the apple to the left of both hands. Despite not having encountered this setup during training, the model successfully places the red apple into the basket via a two-handed handover, albeit with jerkier motion.

Bjorck et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. arXiv. 2025.

GR00T N1: increasing the number of fine-tuning demonstrations

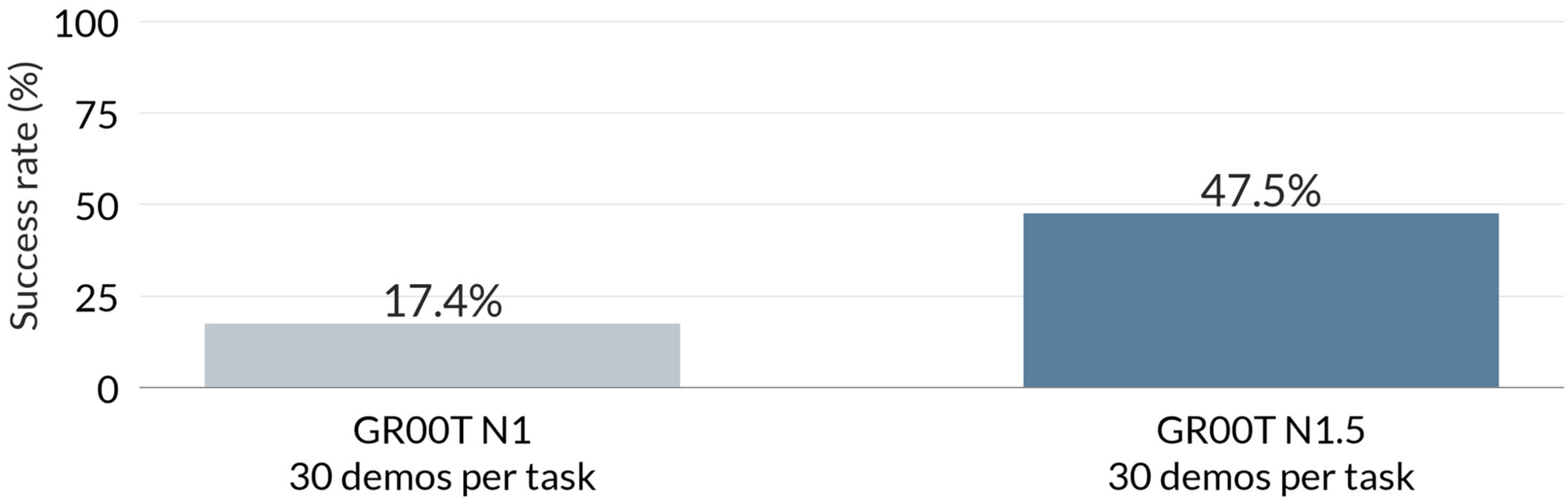
- With 10% of the target robot data, N1 already gets 42.6% average success
- A single-task diffusion policy gets to only 46.4%; whereas N1 with all target data reaches 76.8%
- Increased success is *postulated to be due to* recovery behaviors and vision-action priors learned at scale



Bjorck et al. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. arXiv. 2025.

GR00T N1.5: data-limited post-training

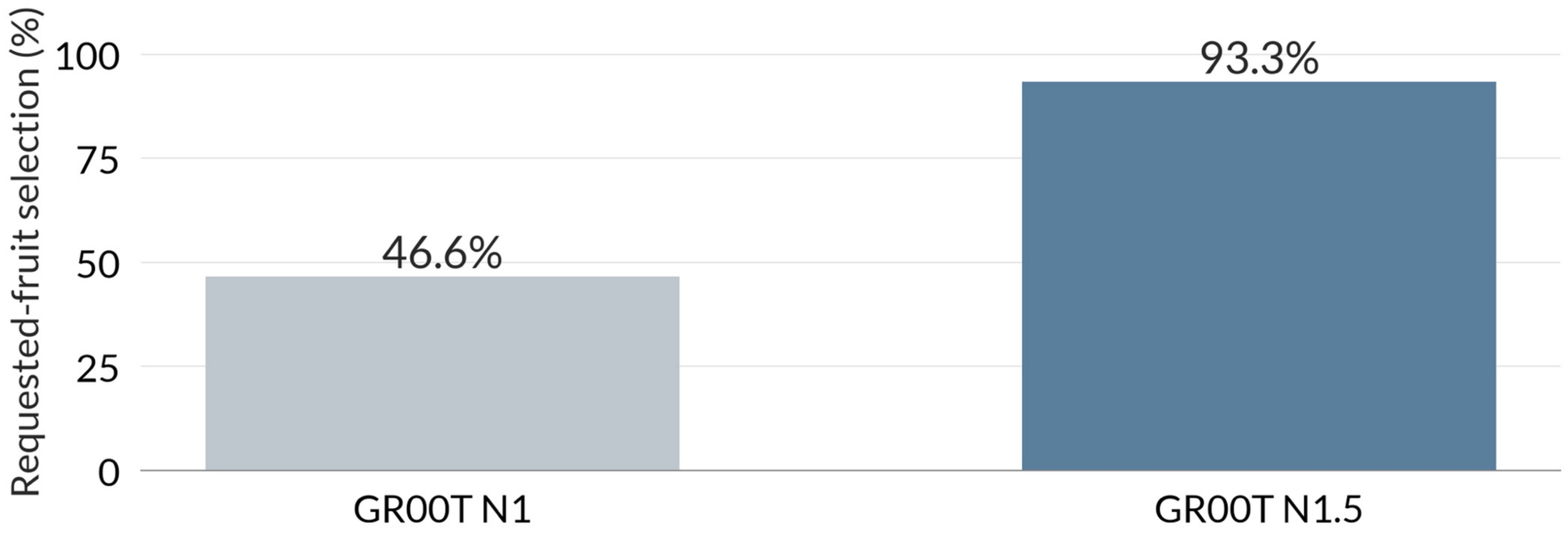
- N1.5 is significantly better in the low-data regime (~30 demos per task)
- At 30 demonstrations per RoboCasa task: N1 reaches 17.4%, N1.5 reaches 47.5%



NVIDIA. GR00T N1.5. Technical release. 2025.

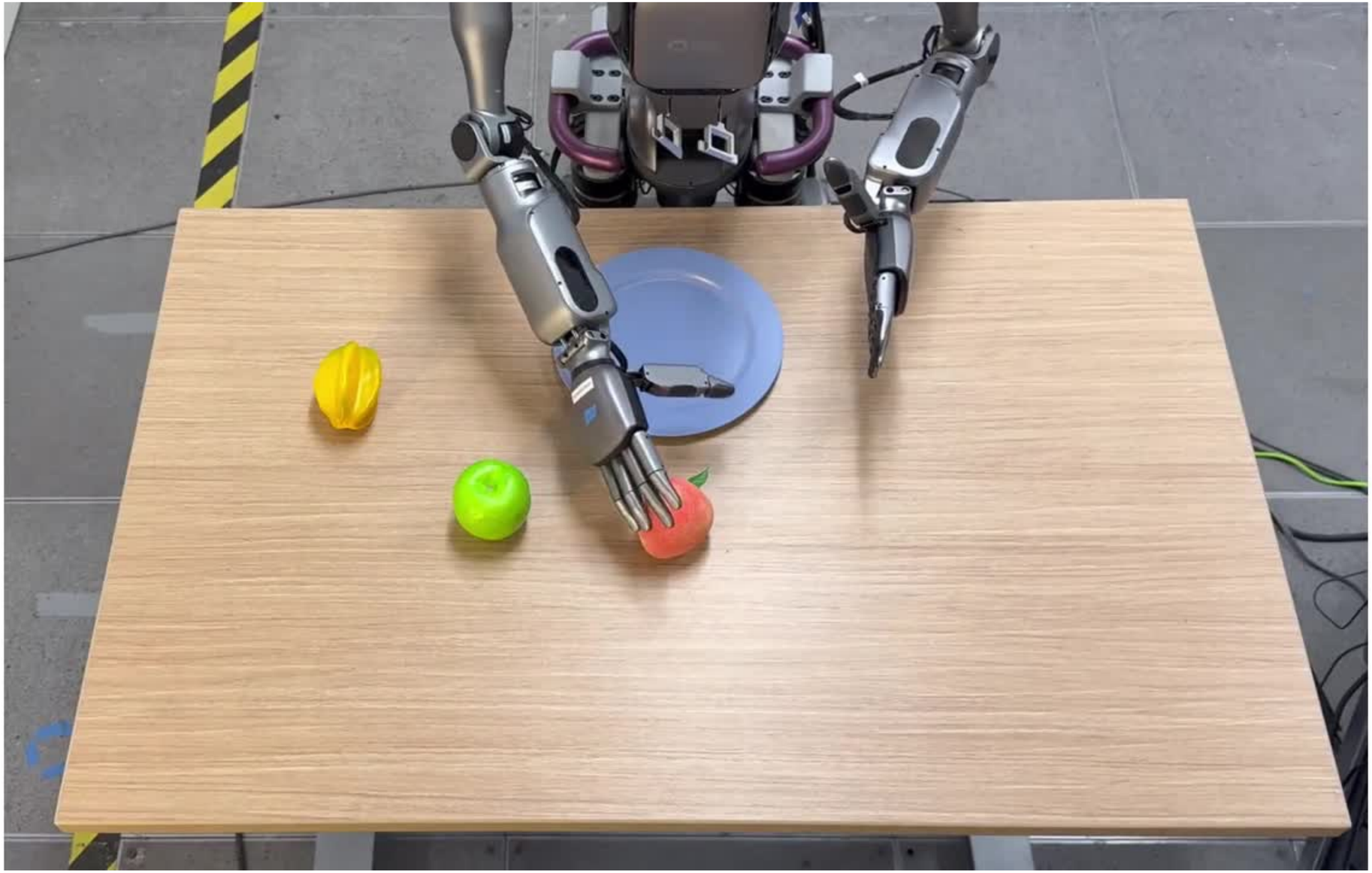
GR00T N1.5: language following on a GR-1 humanoid

- Language following task: 2 fruits are on a table; robot is asked to place one of them onto a plate
- N1.5 significantly improve language command following ability



NVIDIA. GR00T N1.5. Technical release. 2025.

GR00T N1.5: language-following on a GR-1 humanoid

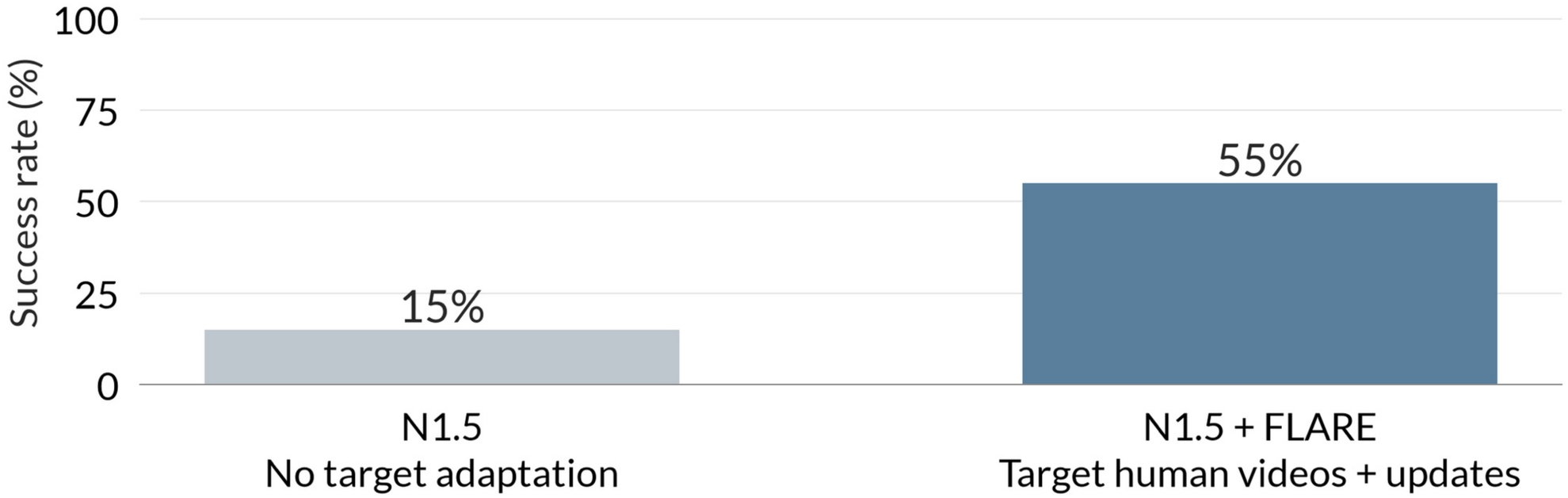


NVIDIA. GR00T N1.5. Technical release. 2025.

*If you are a GR00T N1.5 author / team member, and would like this corrected, please reach out to kmj@jhu.edu w/ clarifications and I'll update this slide!

GR00T N1.5: learning from human video

- On ten novel objects, unseen during training, GR00T N.15 is able to adapt without any robot demonstrations, by using human videos
- FLARE (future latent prediction) objective is used when post-training on these videos
- *Caveat**: Unsure how many human demos were used (guess ~150), and if any robot data was at-all collected (guess ~10)



NVIDIA. GR00T N1.5. Technical release. 2025.
Zheng et al. FLARE: Robot Learning with Implicit World Modeling. CoRL. 2025.

GR00T N1.5: Learning to manipulate novel objects from human videos





Demonstrations using a novel object, captured on a GoPro (left) and the GR-1 robot (right)

NVIDIA. GR00T N1.5. Technical release. 2025.

Summary: how do we retain web-scale pretrained knowledge?

- (At least) 3 approaches
- Finetune the VLM on robot-related data
- Don’t pass action gradients through the VLM backbone (in both pi 0.5 and GR00T N1.5)
- Leverage future observation prediction (a.k.a. world modeling) objectives and pretrained video models as applicable
- We also looked at one of our very first WAM (world-action-model) instantiations without labeling it one

Zitkovich et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. CoRL. 2023.

Driess et al. Knowledge Insulating Vision-Language-Action Models: Train Fast, Run Fast, Generalize Better. NeurIPS. 2025.

Problem 4: learning new tasks efficiently

- We've so far seen examples of VLAs making learning new tasks easier, given finetuning data in the target environment
- What if there's little-to-none such fine-tuning data?
- What if there's zero robot demonstrations to learn new behaviors?
- Ideas:
  - leverage *human demonstrations* that show *what* the task is, and a sketch of *how* to do it;
  - use generative models to generate / augment demonstrations
  - *scale* (more data, more compute, bigger model sizes)
- What we've seen so-far of VLAs is what can be termed *pre-training*. What we'll see from now on is better termed *post-training*.

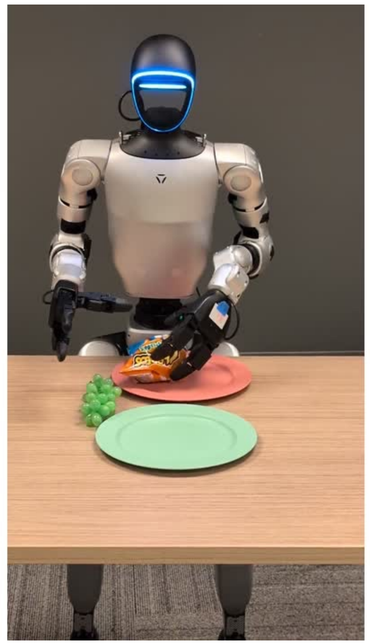
DreamGen: generated training trajectories



Jang et al. DreamGen: Unlocking Generalization in Robot Learning through Video World Models. CoRL. 2025.

GR00T N1.5 and DreamGen: training on *dreamed* data

- Turns out, we can train policies solely on *dreamed* data (i.e., data generated by a video generation model; and never collected on a real robot)
- DreamGen (a text-to-video model + inverse dynamics model to provide action annotations) was used to synthesize 12 new behaviors
- No teleop data; behaviors solely exist in generated data!

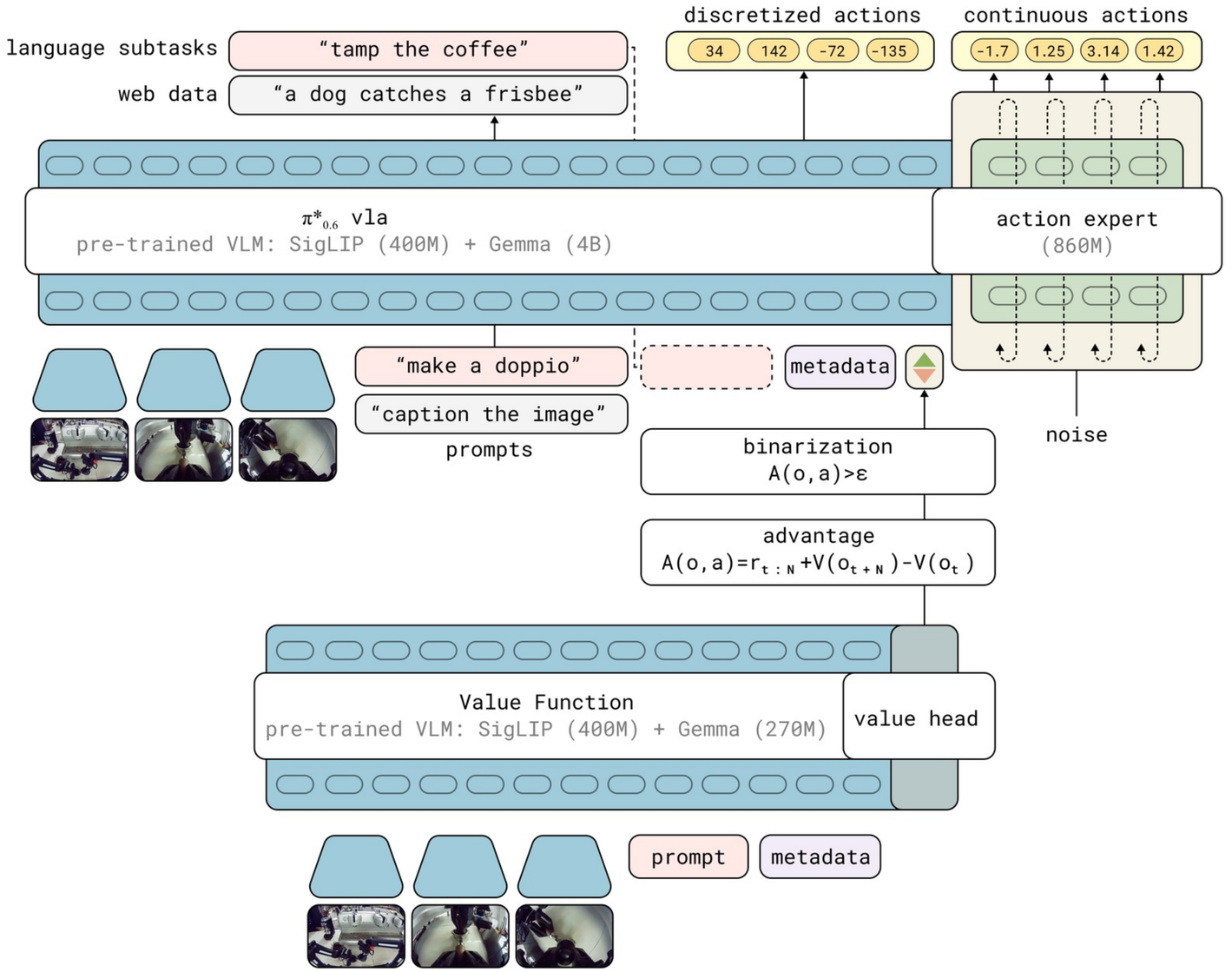


NVIDIA. GR00T N1.5. Technical release. 2025.

Jang et al. DreamGen: Unlocking Generalization in Robot Learning through Video World Models. CoRL. 2025.

*Not an actual *recap* of something we've seen previously in class!

RECAP*: learning from experience (not just via imitation)



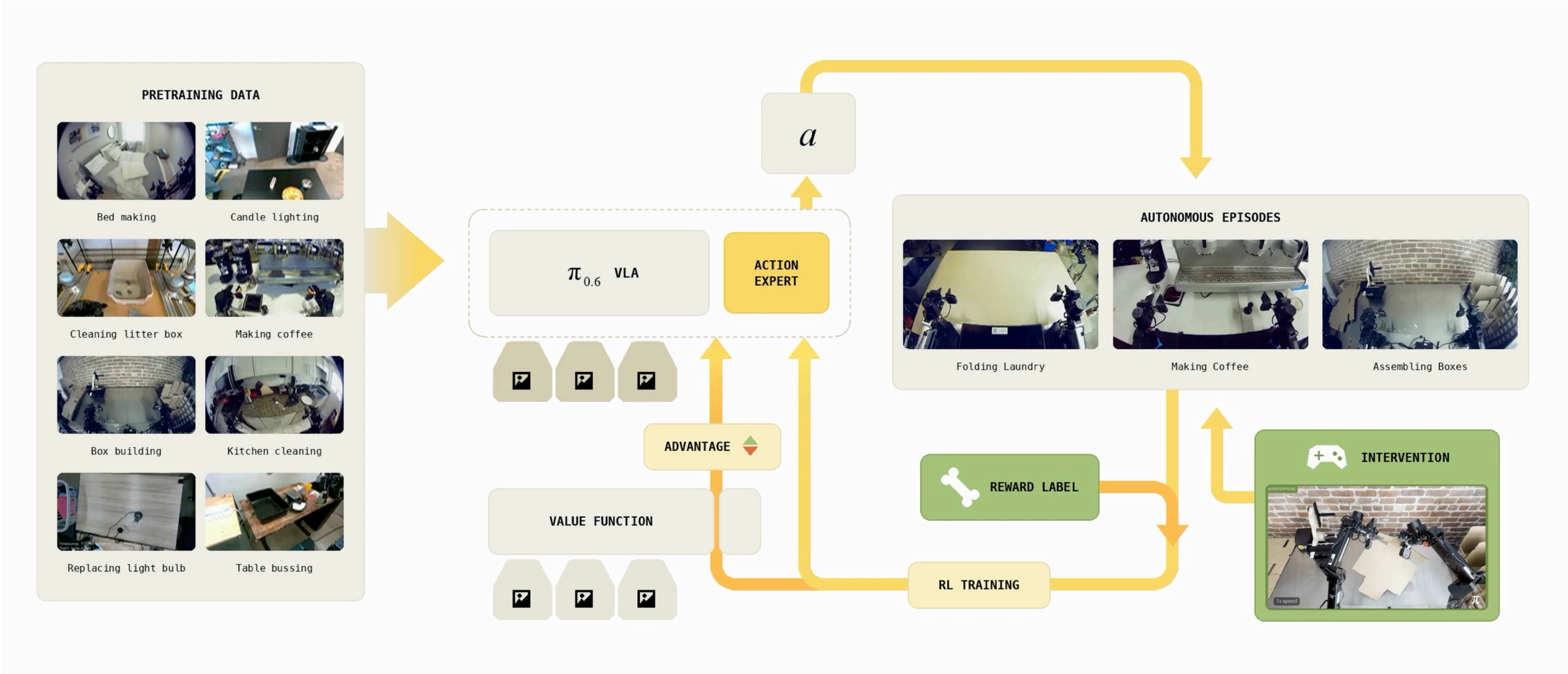
Amin et al. pi0.6*: a VLA That Learns From Experience. RSS. 2026.

pi0.6 + RECAP: motivation

- In naive behavioral cloning, we assume that all actions are expert actions (and are uniformly *optimal*)
- Cloning every action => model also clones mistakes (or sub optimality)
- Instead, compute and use *advantage* (i.e., *was this action better / worse than average* for this situation?)
- RECAP: RL with Experience and Corrections via Advantage-contitioned Policies)
- DAgger is one step better than naive BC, but requires humans to intervene at *just the right moment* a suboptimal outcome may occur
- Instead of human intervention, assign credit (+ve or -ve) to each action, and train a policy that can perform *high-advantage* actions

Amin et al. pi0.6*: a VLA That Learns From Experience. RSS. 2026.

pi 0.6 + RECAP


PRETRAINING DATA
Bed making
Candle lighting
Cleaning litter box
Making coffee
Box building
Kitchen cleaning
Replacing light bulb
Table bussing
a
$\pi_{0.6}$ VLA
ACTION EXPERT
ADVANTAGE
VALUE FUNCTION
AUTONOMOUS EPISODES
Folding Laundry
Making Coffee
Assembling Boxes
REWARD LABEL
RL TRAINING
INTERVENTION

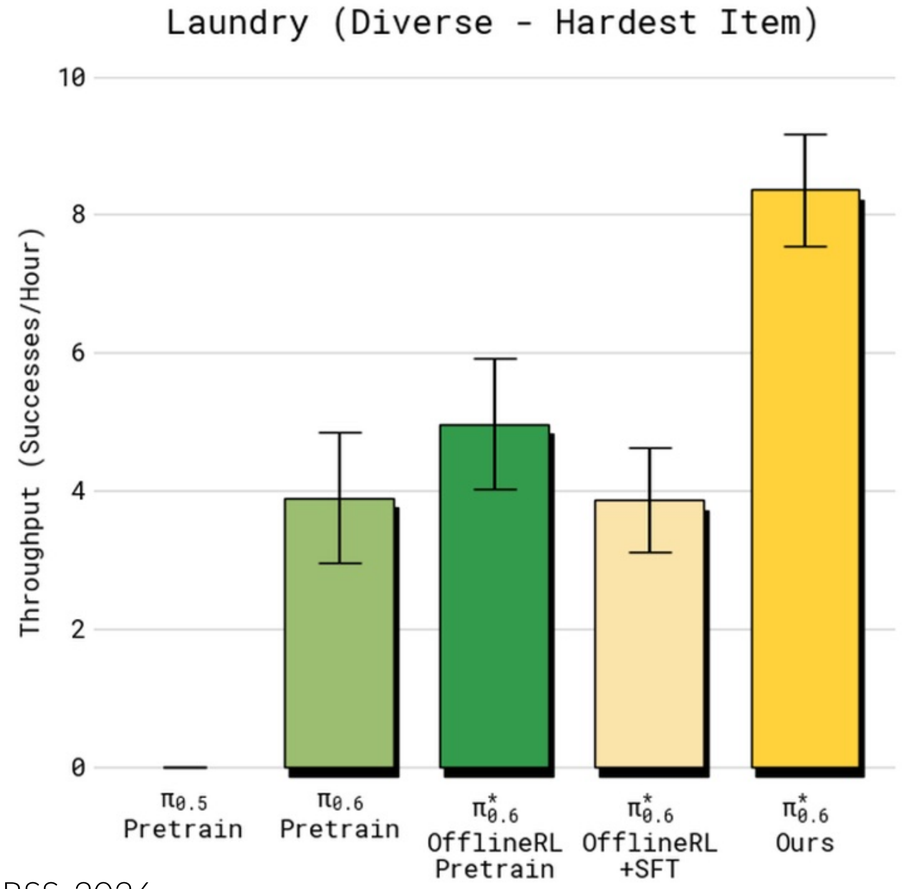
pi0.6 + RECAP: approach

- Learn a value function from recorded episodes (training/rollouts) — use per-step cost, success/failure labels
- Estimate the *advantage* of each training action (did it produce a better outcome than expected, given the observation?)
- Convert advantage estimates into binary labels (task-specific thresholds)
- Action expert is conditioned on advantage labels / weights during training (imp: use both higher and lower advantage labels)
- During evaluation, simply sample actions with higher advantage
- Note: slight difference w.r.t. advantage-weighted regression (AWR), where we weigh training loss based on advantage value of each action. In RECAP, advantage values are passed as conditioning

Amin et al. pi0.6*: a VLA That Learns From Experience. RSS. 2026.

RECAP: reliability and throughput

- Post-training improves both task success and completion time
- Throughput (successes / hour) -> rewards reliability and speed



Amin et al. pi0.6*: a VLA That Learns From Experience. RSS. 2026.

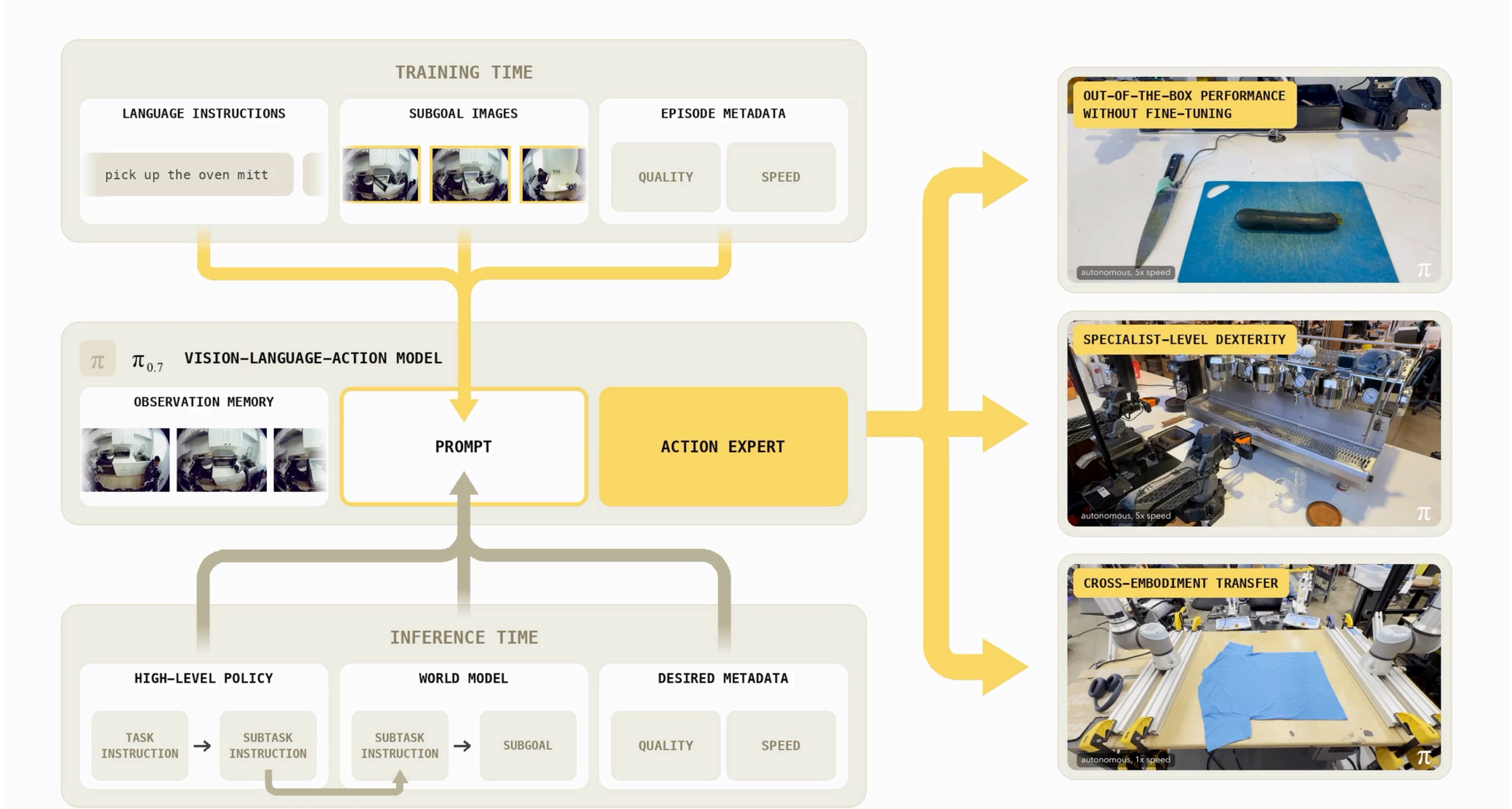
Memory and partial observability

- Most robotics tasks of interest are non-Markovian, and require memory
- MEM combines memory at multiple scales (short-term videos, longer-term text)



Torne et al. MEM: Multi-Scale Embodied Memory for Vision Language Action Models. arXiv. 2026.

pi0.7: sub-goal image generation and conditioning



pi0.7: conditioning on heterogeneous behavior

- Condition on instructions, subgoal images and *other metadata**
- This context helps disambiguate demonstrations, autonomous data and execution strategies



*Physical Intelligence (the company) has not revealed much about what this metadata comprises

Ai et al. pi0.7: a Steerable Generalist Robotic Foundation Model with Emergent Capabilities. arXiv. 2026.

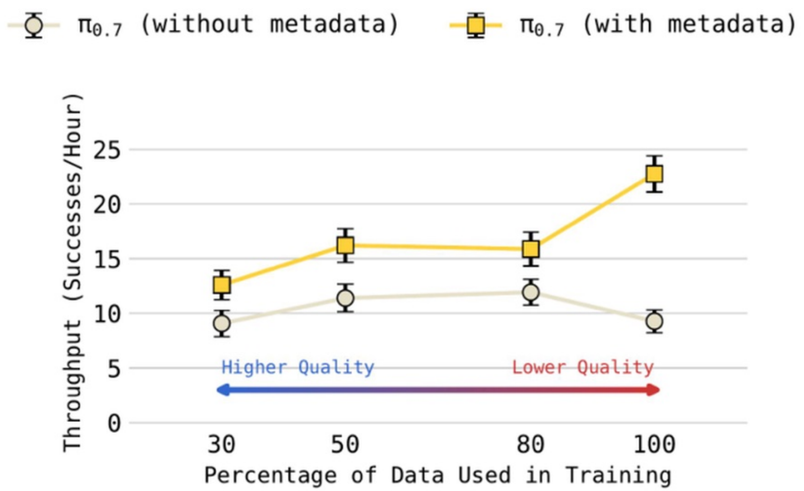
pi0.7: more detailed coaching



Ai et al. pi0.7: a Steerable Generalist Robotic Foundation Model with Emergent Capabilities. arXiv. 2026.

pi0.7: data diversity and conditioning

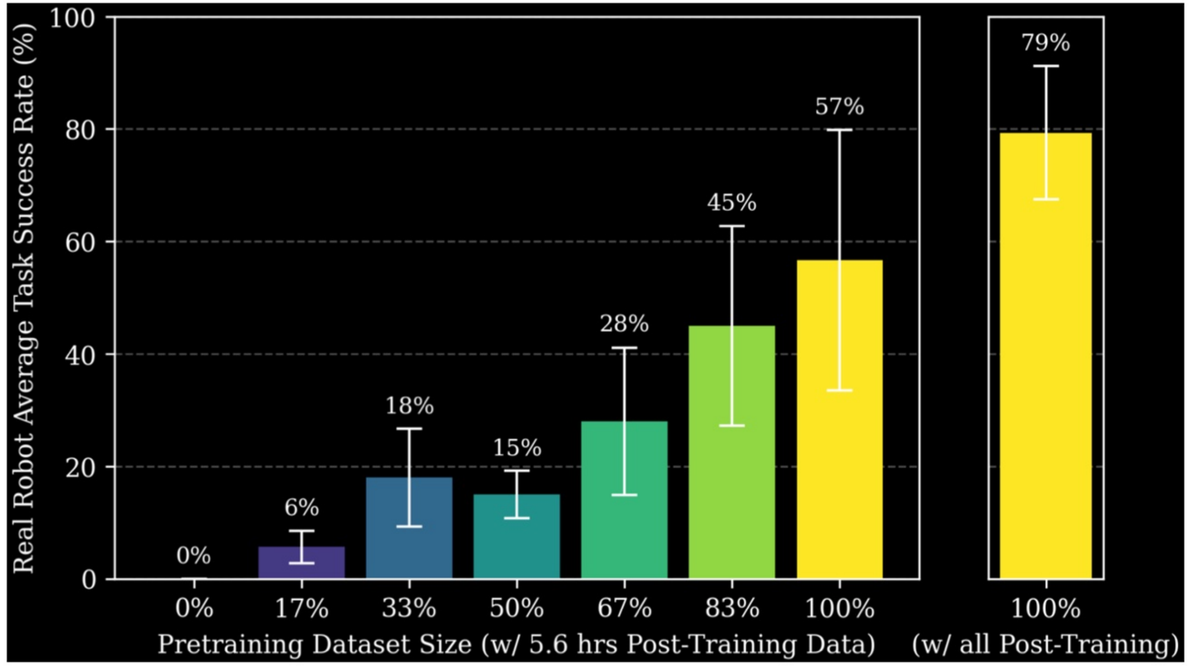
- Removing the most diverse 20% hurts more than removing a random 20% of the data
- Metadata conditioning changes how effectively the policy uses increasingly mixed-quality experience





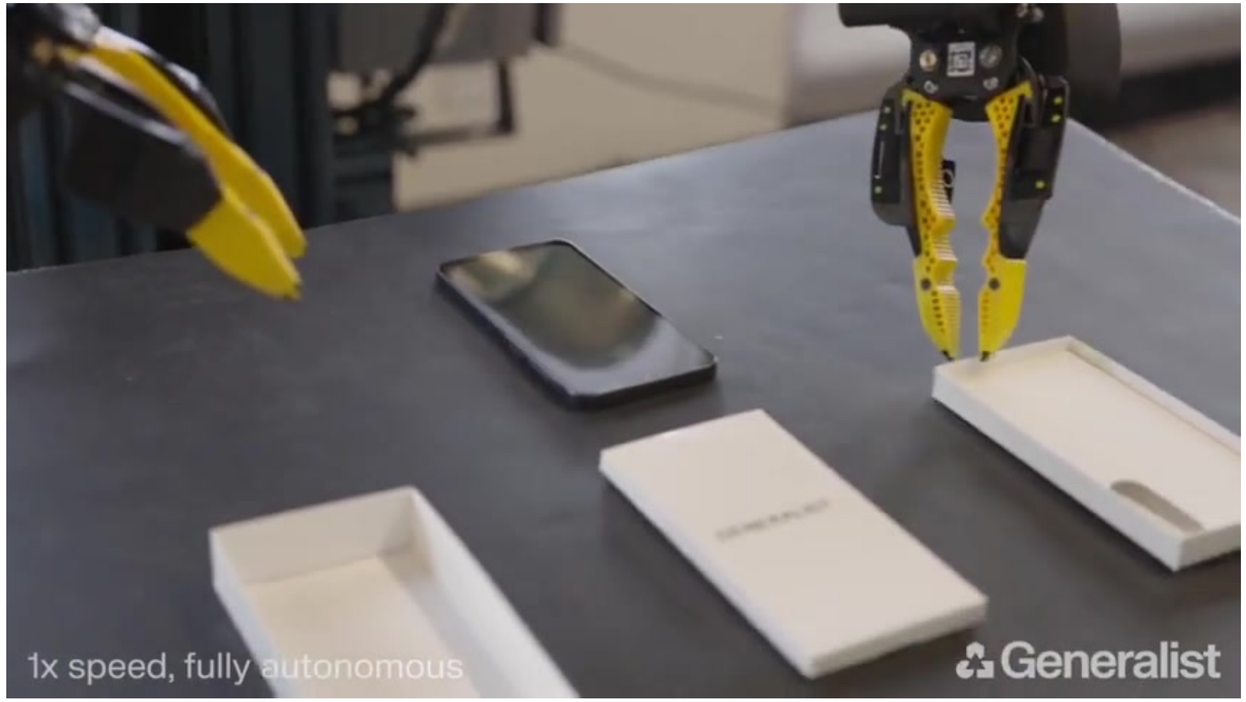
Generalist AI: scaling pertaining w/ UMI-like interfaces (and robot data)

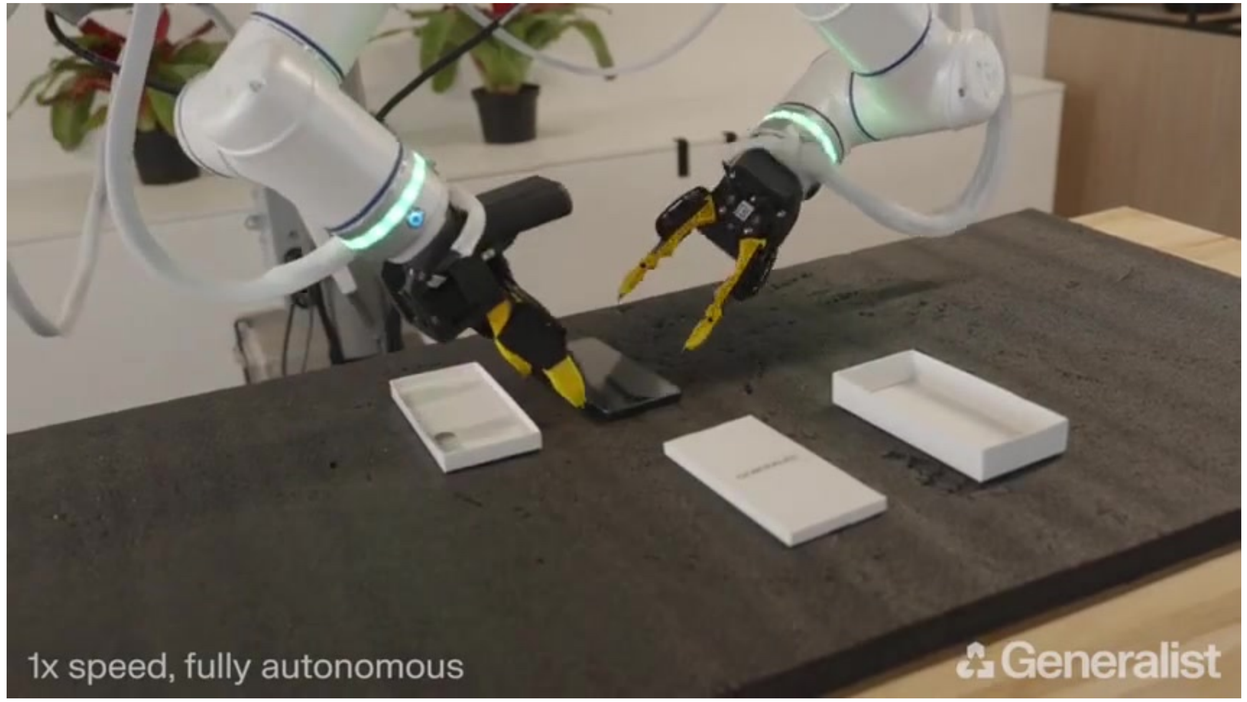
- GEN-0: a 10B+ parameter foundation model; harmonic reasoning (architecture named, but internals weren't revealed, as has become the norm); studied scaling laws
- GEN-1: pretrained from scratch; followed by about an hour of robot data for selected tasks
- GEN-1.5: claims one-shot generalization (one demo that shows the task suffices)



Generalist. GEN-0. Technical release. 2025.
Generalist. GEN-1. Technical release. 2026.
Generalist. GEN-1.5. Technical release. 2025.

GEN-0 vs GEN-1: Packing a smartphone

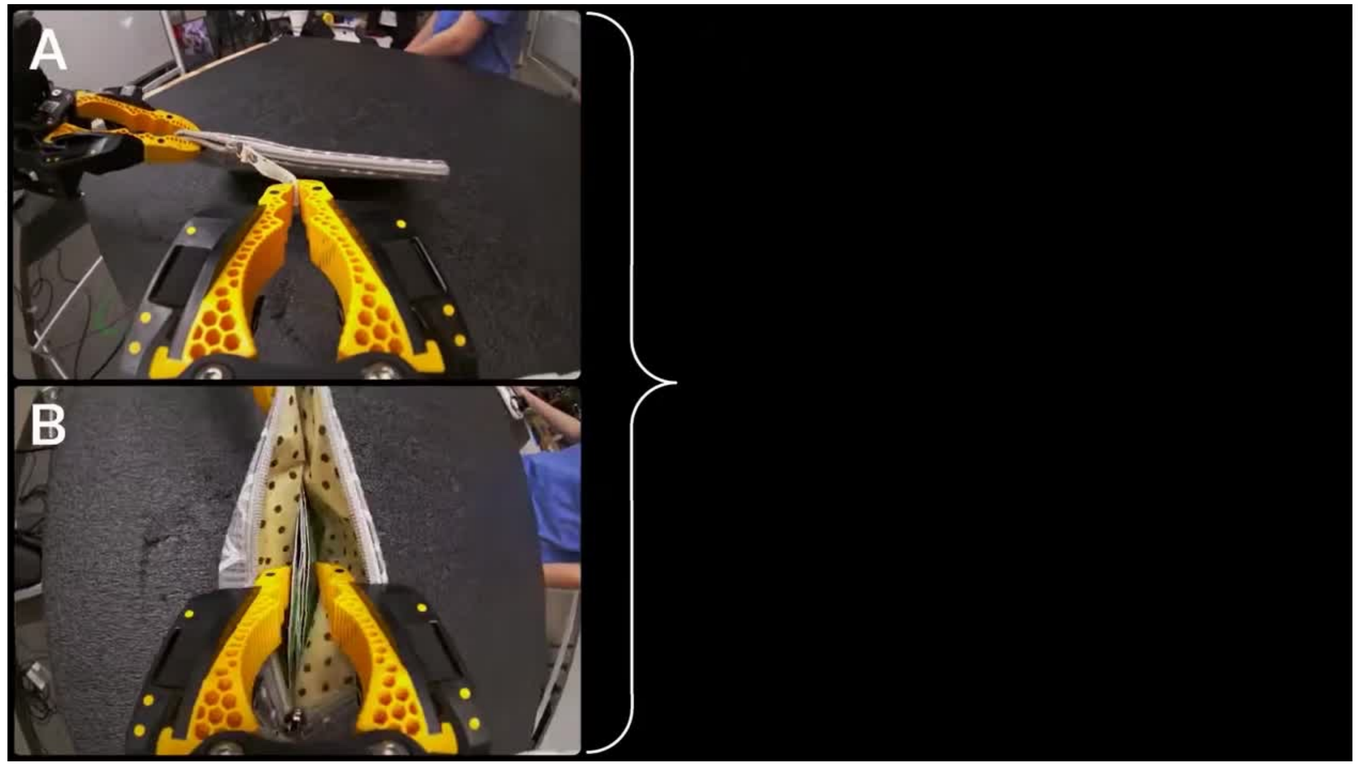




Generalist. GEN-1. Technical release. 2026.

GEN-1.5: one-shot transfer

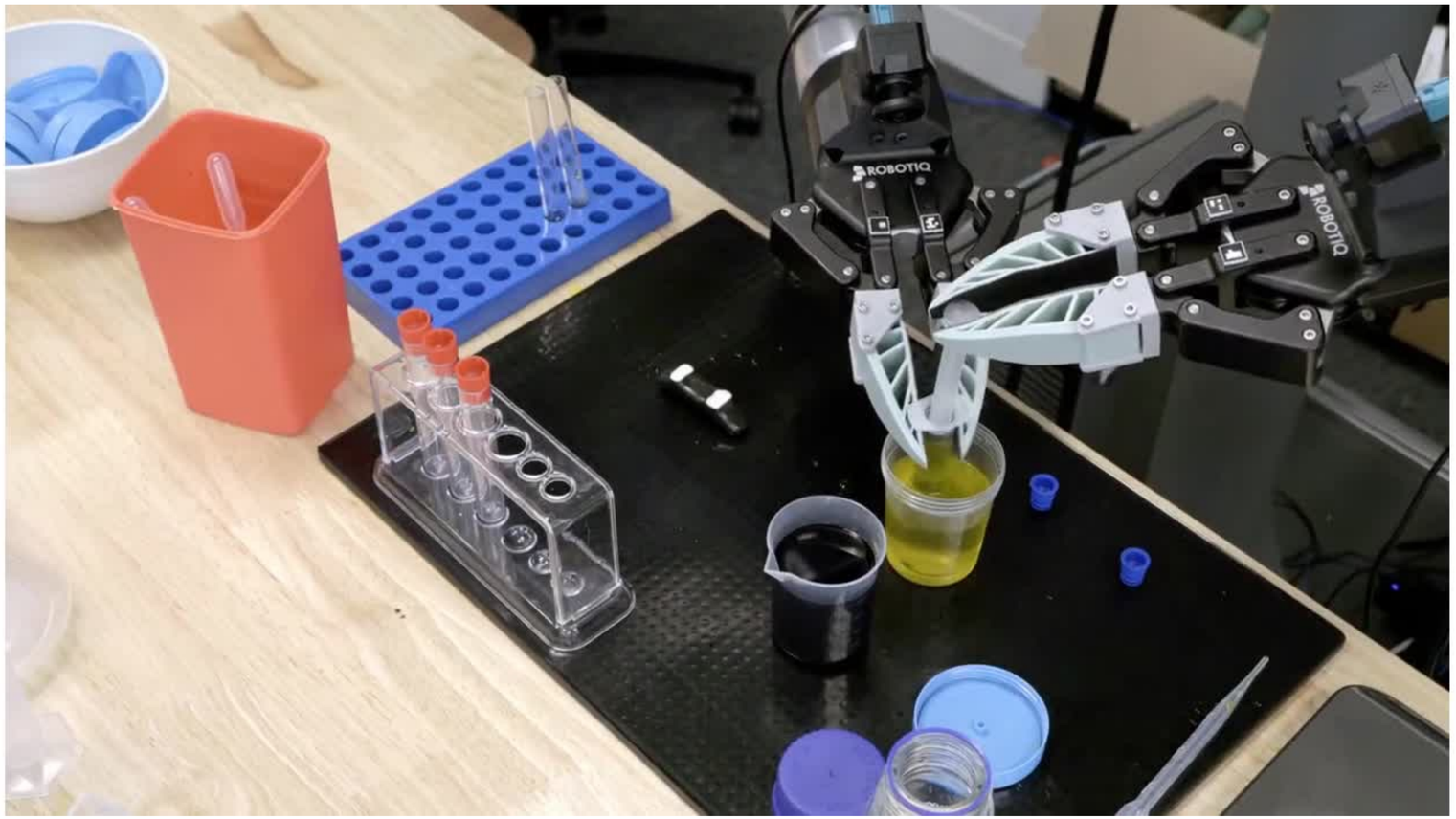
- The *one-shot* physical prompts contain sensor observations and action trajectories
- Demos are *reportedly* fed in-context and executed with fixed weights



Generalist. GEN-1.5. Technical release. 2026.

Skild S1: video-conditioned execution

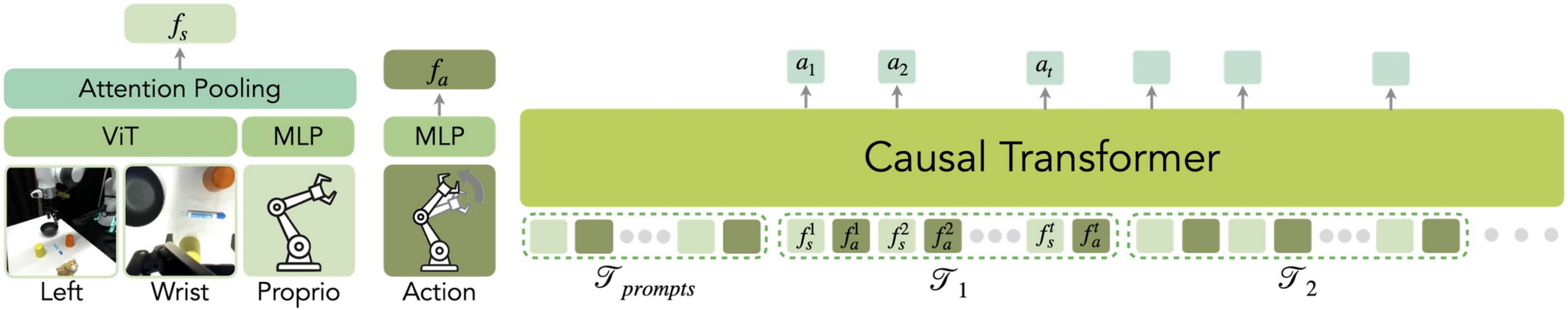
- Reports held-out task execution from a *video* prompt (w/ fixed policy weights)



Skild AI. Skild S1. Technical release. 2026.

Demonstration-conditioned policies

- (Some academic research that's more transparent in terms of how in-context imitation works)
- ICRT uses a demonstration as context for predicting actions
- Behavior Prompting Policy trains on demonstration/execution pairs from the same task



Fu et al. ICRT: In-Context Imitation Learning via Next-Token Prediction. ICRA. 2025.

Patel et al. What Enables In-Context Behavior Prompting for Manipulation? arXiv. 2026.

Helix 2.5: transfer to new homes

- Adapt a pretrained base model to three behaviors, then evaluate in unseen homes
- Tests environment transfer under behavior-specific adaptation



Figure. Helix 2.5: Zero-Shot 30-Home Generalization. Technical release. 2026.

What changed in these (and other) recent results?

- Broader task coverage, better context can make heterogeneous experience more useful
- Video prediction objectives; memory (direct / indirect) help long-horizon and/or non-Markovian tasks
- Data and model sizes; scaling laws
- But above all, *attention to detail* and *good full-stack engineering*